

Agentic Graph Reasoning: Claim-Level Structural Verification for Multi-Hop Knowledge Graph Question Answering

Md. Sakif Khan^{a,*}, Sadia Sharmin^a

^aDepartment of Computer Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh

Abstract

Multi-hop question answering over knowledge graphs composes facts across several edges. A system that retrieves once before reasoning cannot know what the second hop will need. Agentic approaches interleave retrieval with reasoning, but none checks that what the final answer asserts is supported by what the agent retrieved. We present Agentic Graph Reasoning (AGR), an explicit state machine over a constrained graph tool API. Its structural verification layer decomposes each draft answer into atomic claims, checks them against the traversed triples before emission, and attaches the matching triples as evidence. On 400 certified questions from each of WebQSP and ComplexWebQuestions, over a Freebase-derived environment, AGR reaches Hits@1 of 0.755 and 0.522 against four baselines that share one frozen backbone and one call budget, using roughly half the language-model calls of the strongest. A component-level ablation with paired significance testing finds that removing the planner *improves* accuracy on the shallower benchmark

*Corresponding author

Email address: sakifkhanopu@gmail.com (Md. Sakif Khan)

while cutting tokens by 31%. Three of four components, claim verification among them, show no detectable accuracy effect. A hand-read census of all 259 residual failures identifies the *echo attractor*: a true, grounded entity one hop from the answer that passes every grounding check and that consensus-based evaluation cannot detect.

Keywords: knowledge graph question answering, large language models, agentic retrieval, ablation study, hallucination, benchmark quality

1. Introduction

Large language models answer factual questions fluently. Often enough, they answer them wrongly. The errors are systematic rather than random, and they are hard to catch from the output alone, because a fabricated answer reads in the same register as a correct one. Huang et al. [1] treat this as a consequence of how the models are built, not as an incidental defect to be patched away. The problem sharpens once an answer requires composing several facts rather than recalling one. Errors compound: a wrong resolution at the first hop rarely announces itself downstream. And a correctness check on the final answer never sees the intermediate steps. So a system that answers correctly by accident looks the same as one that answers correctly by sound reasoning.

Retrieval-augmented generation [2] addresses the first half of this problem by grounding the model in retrieved text. It does not address the second. Retrieval runs once, before reasoning begins, so a multi-hop question must be answered from whatever that single query returned. The passage holding the *intermediate* entity need not resemble the original question at all, and

the question is the only signal the retriever can rank on. Representing the corpus as a knowledge graph [3] turns composition into a traversal rather than an inference over prose. GraphRAG [4] exploits this, but it still retrieves statically. A neighbourhood fetched in one shot cannot know what the next reasoning step will need. It is either too small to contain the answer or so large that the relevant edges are buried.

Agentic systems close that gap by interleaving the two steps. The model takes one step, observes what it found, and decides where to look next [5, 6, 7]. Retrieval becomes navigation, and the system becomes an agent operating on the graph rather than a pipeline reading from it. Section 2 traces that line of work.

One property survives this progression unchanged. Every system named above emits whatever its final generation call produces from the traversed context. Nothing checks that what the answer *asserts* is supported by what the agent retrieved. The gap is not fabrication. Our measurements confirm that graph-navigating systems assert entities that exist in the graph and are reachable along the paths they walked (Section 5.6). The gap is precision. An entity can sit in the retrieved subgraph, have been retrieved correctly, and still be the wrong answer: an intermediate node from the reasoning chain, a container where the question asked for a member, or one of several plausible neighbours chosen arbitrarily. A system that verifies retrieval but never verifies assertion cannot catch any of these.

This paper presents **Agentic Graph Reasoning** (AGR), a knowledge-graph question-answering framework built as an explicit state machine over a constrained, deterministic graph tool API. The program orchestrates the

loop. The language model is consulted only at bounded decision points. AGR’s distinguishing component is a *Structural Verification Layer*. Before an answer is emitted, the draft is decomposed into atomic claims, and each claim is checked against the triples the agent actually traversed. The word *structural* bounds that check as much as it names it. The two graph routes ask whether an edge joins a claim’s endpoints, not whether it is the edge the claim asserts. So a claim that X is Y’s mother is certified by an edge recording that X is Y’s child, and only the entailment fallback reads the relation. Section 8.1 reports the exposure that follows. Claims that cannot be grounded trigger targeted re-exploration or are removed. The claims the traversal itself grounds carry their triples back with the answer.

We are precise about what that layer buys, because the ablation of Section 6 is precise about what it does not. Claim verification does not measurably change how often AGR is right. It changes how often AGR is right *for a reason it can exhibit*. The contribution is the output contract, an answer together with the traversed evidence for it, rather than an accuracy gain. Section 6 reports the null that establishes this at the same length as the effects it does detect.

On 400 certified questions from each of WebQSP [8] and ComplexWebQuestions [9], over a Freebase-derived environment of 2.59 million entities and 8.31 million triples, AGR reaches Hits@1 of 0.755 and 0.522. Four baselines share one frozen backbone and one call budget with it. AGR leads all four, at roughly half the language-model calls of the strongest. On ComplexWebQuestions it is the only system whose accuracy rises with hop count. Its advantage over the agentic baseline is located rather than asserted.

On the questions where the shared cap lets that baseline finish, the baseline is slightly ahead. The entire aggregate margin comes from the 29% and 44% of questions on which the cap truncates it.

Contributions. This paper contributes:

1. **AGR**, a state-machine agent over a constrained graph tool API whose answers carry the traversed triples that support them, so the reasoning is auditable at emission rather than merely asserted (Section 3). Section 8.1 bounds how far that reaches.
2. A **Structural Verification Layer** that checks atomic claims against traversed adjacency before emission. Its repair routes and their firing rates are reported as they behave rather than as designed (Section 3.2).
3. A **controlled comparison** against four baselines on one frozen backbone under one call budget, so that differences are attributable to architecture rather than to model capacity (Section 5). Beside it we report a direct comparison against the published figures of the system whose data distribution we share, and that comparison does not favour this work (Section 5.4).
4. A **component-level ablation** with paired significance testing. It finds that removing the planner *improves* accuracy on the shallower benchmark while reducing token use, and that three of four components show no detectable effect (Section 6).
5. A **hand-read census** of all 259 remaining failures. It names the *echo attractor*, a true, grounded entity one hop from the answer that every grounding check passes, and it quantifies label defects in both benchmarks (Section 7).

Section 2 positions AGR against prior work. Section 3 specifies the framework and the verification layer. Section 4 describes the environment and the experimental protocol, and Section 5 reports the measurements. Section 6 attributes them to components, Section 7 reads what remains, and Section 8 states the limits.

2. Related Work

Three lines of work bear on question answering over knowledge graphs. We take them in the order the problem forces: what retrieval fixed before reasoning can reach, what interleaving retrieval with reasoning buys, and what checking an answer against evidence adds. A fourth subsection names something none of them establishes: which component of an agentic system is responsible for its behaviour. That gap is what this paper addresses.

2.1. *Static Retrieval over Graphs*

Retrieval-augmented generation [2] attaches a language model to an external corpus. The system embeds the question, retrieves lexically or semantically similar passages, and lets the model answer from them. Where the answer sits in a single retrievable passage this works well. Where it must be composed across passages, the ranking signal measures the wrong thing. The passage carrying the intermediate entity resembles neither the question nor anything the system yet knows to look for. Surveys of the graph–language-model intersection identify this as the central limitation of flat retrieval [10].

Knowledge graphs make the missing structure explicit. They store facts as typed edges between identified entities, so composition becomes a traversal

rather than an inference over prose. GraphRAG [4] builds a graph over a corpus and retrieves structured context in place of passages. Related systems extend the idea to long-term memory [11] and to temporally-qualified facts [12]. All of them retrieve *statically*: a subgraph is fetched around the linked entities in one shot and handed to the model. Retrieval runs first and runs once. So it cannot know what the second hop will need, and a fixed-radius neighbourhood is either too small to contain the answer or so large that the relevant edges are lost among thousands of irrelevant ones.

2.2. Agentic Navigation

Interleaving retrieval with reasoning removes that constraint. The model takes one step, observes what it found, and decides where to look next. Retrieval becomes navigation, and the system becomes an agent operating on the graph rather than a pipeline reading from it. This inherits the tool-use and self-reflection patterns established for language agents generally [13, 14, 15].

Think-on-Graph [5] established the pattern for knowledge graphs with model-guided beam search over relations and entities. Plan-on-Graph [6] added two mechanisms to it: decomposition into sub-objectives, and backtracking driven by reflection. Reasoning-on-Graphs (RoG) [7] takes a different route. A fine-tuned model generates grounded relation paths as explicit plans, and the system then traverses them. Because its answers are read off retrieved paths, they are faithful by construction. RoG is also the one surveyed system whose data distribution this paper shares: the knowledge environment of Section 4.1 is built from its published subgraphs. Section 5.4 therefore compares the two directly, published figures and all. Other systems

add tool abstractions [16], cope with incomplete graphs [17], or generalise across structured sources [18]. Recent surveys treat the convergence as a research programme in its own right [19, 20].

KBQA-o1 [21] is the closest system to AGR’s search strategy, and the most important to distinguish from it. It performs agentic knowledge base question answering with full Monte Carlo Tree Search: a policy model proposes actions, a reward model scores them, and rollouts propagate their values back through the tree. That search wraps a ReAct-style agent generating logical forms against Freebase, and the exploration also produces annotations for incremental fine-tuning. AGR does none of this. Its search is guided best-first with a bounded beam and backtracking. It carries neither rollouts nor value backpropagation. The two are separable on that basis, and we name the difference rather than blur it.

2.3. Verification and Self-Correction

A parallel line checks generated content after the fact. Chain-of-Verification [22] has a model draft a response, plan verification questions about it, answer those independently, and regenerate. Multi-agent debate [23] reaches a related effect by having several instances critique one another. Both establish the premise this paper’s verification layer rests on: decompose a draft, check its parts, and factual error goes down. But both check against the model’s own knowledge rather than an external symbolic source, and the limits of models correcting themselves without such a source are documented [24]. Hu et al. [25] demonstrate a self-correcting agentic graph pipeline in a clinical setting. FActScore [26] decomposes generations into atomic facts for evaluation, which is the same decomposition applied as

a metric rather than as a control step.

None of them provides a claim-level check against the *traversed* triples, performed *before* the answer is emitted, inside a graph-navigating agent. That is the gap Section 3.2 fills.

2.4. The Attribution Gap

A second gap is methodological. Agentic systems are assemblies of decomposition, scoring, backtracking, and self-correction, and the literature reports them as wholes. When such a system outperforms a static baseline, the published work rarely establishes which mechanism produced the gain, or whether some contribute nothing. Later work then inherits the entire assembly on faith, along with its cost.

The exception is instructive. KBQA-o1’s own ablation removes MCTS while leaving every other component intact, and GrailQA F1 drops from 78.5 to 48.5 [21]. That is a thirty-point attribution to a single mechanism, obtained precisely because the authors ablated rather than reporting the assembly whole. Section 6 applies the same instrument to AGR. It reports the components for which it detects nothing at the same length as the one for which it does.

3. The AGR Framework

AGR inverts the usual agent pattern. It does not hand the model a set of tools through native function-calling and let the model drive the loop. Instead, a program orchestrates every step, calls the graph tools deterministically, and consults the model only at bounded decision points, through prompts constrained to return JSON. Three consequences follow.

Every graph operation is logged by construction rather than reconstructed from the model’s account of what it did. Budgets are enforced by counters a router checks, not requested in a prompt the model may ignore. And a malformed model response degrades one decision rather than corrupting control flow: every call is wrapped in a parser that, on failure, spends one metered retry asking the model to repair its own output before falling back. The rest of this section specifies the loop that produces a traversal (Section 3.1), the layer that checks a draft against it before an answer is emitted (Section 3.2), and the budgets that bound the whole loop’s cost (Section 3.3).

3.1. State Machine and Tool API

Figure 1 gives the topology: six agent nodes over a shared typed state, with three cycles. The **Planner** decomposes the question into an ordered chain of sub-objectives in one model call. **Explorer** and **Evaluator** form a search loop. The explorer expands the frontier, and the evaluator judges whether the active sub-objective is satisfied. The **Backtracker** restores an earlier, higher-scoring frontier and bans the edges that led away from it. That ban list is what turns backtracking into a search operation. Without it, a deterministic scorer facing the same restored frontier would select the same edges again. Two properties of the topology matter beyond the nodes themselves. It contains cycles rather than running once through, because exploration continues until the evaluator is satisfied and verification can send the agent back to explore. And every path from Evaluator to Answerer passes through the **Verifier**. No route reaches the Answerer without it. That routing is the structural expression of this paper’s contribution and the

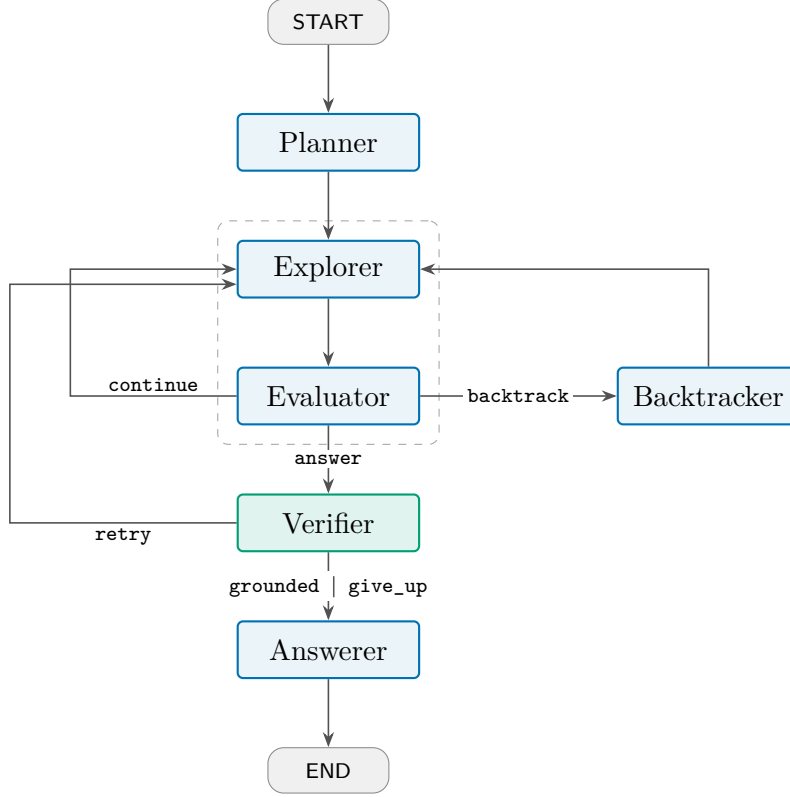


Figure 1: The AGR state machine. The dashed region is the Explorer $\leftrightarrow$ Evaluator search loop. Evaluator $\rightarrow$ Backtracker $\rightarrow$ Explorer restores an earlier frontier, and Verifier $\rightarrow$ Explorer is verification-driven re-exploration. The budgets of Section 3.3 bound all three cycles. Every path from Evaluator to Answerer passes through the Verifier.

subject of Section 3.2.

Five graph operations back this loop. They are parameterised Cypher templates, not free-form queries the model would write itself. A generated-query failure is indistinguishable, in the results, from a reasoning failure, and this paper’s central claims are about reasoning and verification, not about syntax. Only four of the five are reachable from a node in the final design.

`verify_triple` was built first, to check a claim $\langle h, r, t \rangle$ by asking whether that exact relation held between the two entities. It did not survive contact with generated text. Claim decomposition phrases relations in English (“played for”, “is the capital of”), and those phrases rarely correspond to a single Freebase predicate, so matching on them rejected claims that were true. `verify_connection` replaced it. It drops the relation and asks only whether the two entities are adjacent, directly or through one mediator node. That is the check Section 3.2 actually uses. `verify_triple` remains in the API and in its unit tests, but nothing in the running loop calls it.

Each candidate edge the Explorer considers is scored by

$$\text{score}(c) = \alpha \cos(v_{\text{rel}(c)}, v_{\text{obj}}) + (1 - \alpha) \text{LLM}(c),$$

which blends two signals at the frozen $\alpha = 0.7$. The first is the cosine similarity between the candidate relation’s embedding and the embedding of the *active sub-objective*. The objective, not the original question, is what gets embedded, and that substitution is the concrete mechanism through which decomposition is meant to help. The second is a batched model judgement over a pre-filtered shortlist of twenty candidates. The blend is applied after the model call rather than inside the prompt. A prompt free of any configuration value lets an α sweep reuse one set of cached responses across every condition. Setting $\alpha = 1$ disables the model term entirely, which is the ablation Section 6 reports.

3.2. Structural Verification Layer

The navigation loop above ends with a set of traversed triples and a set of accumulated candidate entities. Nothing in it constrains what the final

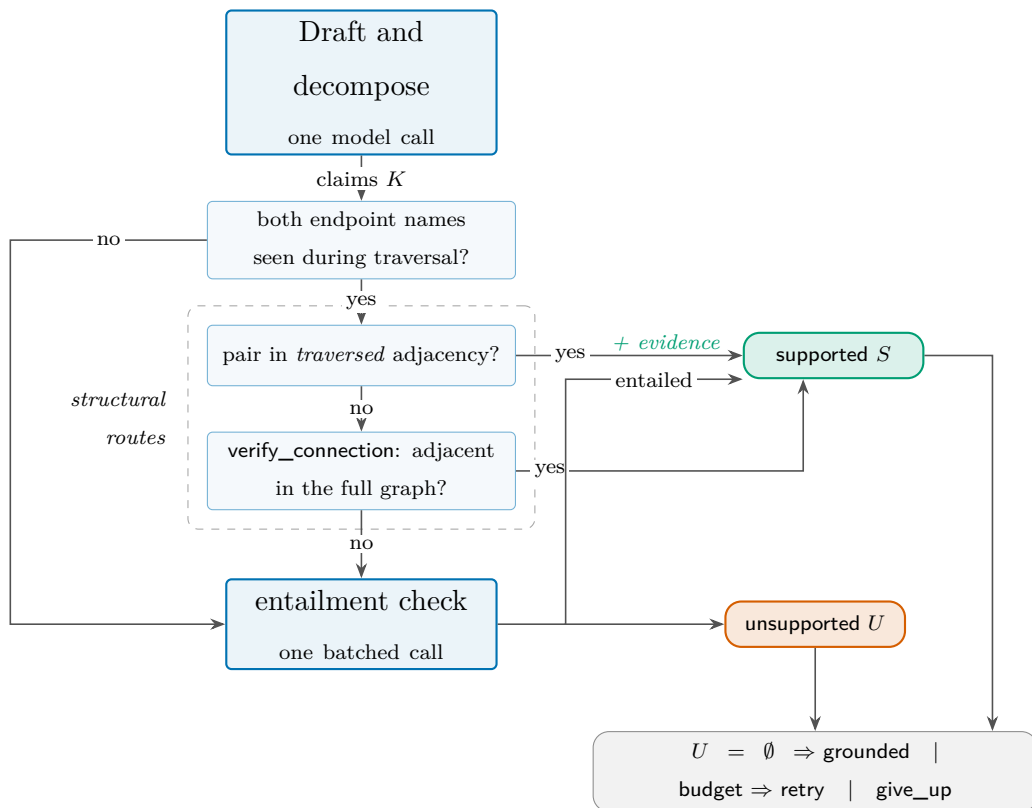


Figure 2: The path one atomic claim takes through the verification layer. Three tests are tried in a fixed, cheap-to-expensive order, and a claim leaves at the first that settles it. Only traversed adjacency attaches citable evidence. A claim naming an entity the agent never reached skips both structural routes entirely.

answer *asserts*. The verifier is the check that does. A single model call drafts prose, extracts the entities the draft asserts as its answer, and decomposes the draft into atomic claims $\langle h, r, t \rangle$ over entity *names*, since the model has never seen a graph identifier. The relation slot is left as free text. Asking the model to name a Freebase predicate would fail as unreliably as `verify_triple` did.

Each claim is routed through up to three tests, and it leaves at the first one that settles it. **Traversed adjacency** collapses the traversed triples into an undirected set of endpoint pairs. A claim is supported if its pair is in that set, and every traversed triple joining the pair is attached as citable evidence. This is the only route of the three that attaches evidence. **verify_connection** catches claims whose endpoints were both reached but never walked together. It queries the full graph for the same relation-agnostic adjacency. Claims that clear neither route, together with those naming an entity the agent never traversed at all, fall to an **entailment check**: one batched model call judging each remaining claim against the same fact window the drafter saw. The order is deliberate. Symbols decide, and the model is consulted only where the graph is silent. Cheap-to-expensive ordering also means the model adjudicates a residue rather than every claim.

The router that follows has three outcomes. No unsupported claim routes to **grounded**, and the draft is emitted verbatim with no further model call. An unsupported claim with verification, depth, and time budget remaining routes to **retry**: the verifier re-targets the agent on the first unsupported claim, and control returns to the Explorer. Otherwise the router takes **give_up**. One rewrite call then restates the draft around a *deterministically filtered* entity list. An entity survives if it appears in a supported claim or in no unsupported claim. An earlier design let the model regenerate the list itself, and that reintroduced exactly the ungroundedness the layer exists to prevent.

The layer’s reach should be stated plainly here, because it bears directly on how the null result of Section 6 should be read. On the 80-question

development set used to validate this design, 77 questions asserted no unsupported claim and took the grounded path at zero additional model cost. The remaining 3 took `give_up`. The retry route did not fire once on that set, because every question that reached the verifier with an unsupported claim had already exhausted its depth budget by the time it got there. On most questions, then, the layer is a pass-through that certifies a draft rather than a mechanism that visibly intervenes. Sections 5 and 6 report what it measurably changes at test scale. Engineering detail beyond that, including defects found along the way and recorded rather than quietly repaired, is in the repository rather than reproduced here.

3.3. Budgets and Termination

Table 1 lists the caps that bound every run. Routers and the model client enforce them; no prompt requests them. Three of them (depth, wall-clock, and verification iterations) are checked at two separate sites each, because the `retry` edge re-enters the Explorer without passing back through the router that would otherwise catch it. Closing that gap is what lets Table 1 be called a guarantee rather than a convention. The two rows below the rule are not budgets but tool-layer caps. They bound what the agent can see rather than what it can spend, and Section 4.2 shows why they matter for the comparison.

Every cycle in Figure 1 therefore passes through a router that checks or decrements a monotone counter. So no sequence of model outputs, however adversarial, can produce a non-terminating run. The cost of enforcing this is small in the common case. The grounded path above adds nothing beyond the single drafting call already required to produce a draft. At test scale, removing the layer altogether saves 2% and 6% of tokens on the two ablation

Table 1: Budget configuration. Every row above the rule is hashed into the run configuration, so a reported result can always be traced to the caps that produced it. The two rows below the rule are tool-layer caps that bound retrieval rather than spend.

Parameter	Value	Enforced in
max_depth	4	post-evaluation <i>and</i> post-verification routers
beam_width	3	Explorer (per-anchor slice)
max_anchors	5	Explorer <i>and</i> Evaluator (frontier slice)
max_backtracks	3	post-evaluation router
max_llm_calls	25	model client, before every call
max_verify_iters	2	post-verification router <i>and</i> the verifier
max_seconds	300	post-evaluation <i>and</i> post-verification routers
max_relations	300	get_relations: candidate relations offered to the scorer, sorted by fanout
max_fanout	200	get_neighbors: neighbours returned per expansion

half-splits (Table 6). Whatever the case for or against the layer, it is not a cost argument.

4. Experimental Setup

Everything in this section exists to make one inference safe: that a difference between two systems is a difference between architectures. We describe the environment all five systems share first, then the benchmarks,

the four baselines, and the single frozen backbone every one of them calls, and finally the metrics and the protocol fixed before any test question was run. Where something is *not* held constant, we name it here rather than leave it for a reader to find.

4.1. Knowledge Environment

Every dataset this paper consumes is public, standard, and third-party. The Freebase [3] snapshot and the two question sets all arrive through one distribution: the per-question subgraphs published with Reasoning-on-Graphs [7], released as `rmanluo/RoG-webqsp` and `rmanluo/RoG-cwq`. The datasets are used unmodified. What is built here is the environment derived from them: a property graph of 2.59 million entities and 8.31 million triples over 7,058 distinct relation types, assembled as the union of every per-question subgraph across both datasets and all three splits. All five systems navigate that one graph. Just under two-thirds of its nodes (63.7%) are unnamed mediators encoding n -ary facts. That is why the tool layer traverses them transparently rather than exposing them as candidate entities (Section 3).

An environment assembled this way is friendlier than full Freebase, and we state the consequence rather than leave it to be inferred. Before any system ran, a validation gate measured what fraction of questions have at least one gold answer reachable from a topic entity within the four-hop cap: 97.0% on the WebQSP sample and 99.2% on the ComplexWebQuestions sample. That ceiling is why a failure in Section 7 can be attributed to the system rather than to a missing edge. Almost every question is answerable in principle, so a wrong answer is a navigation or selection error, not an absent

fact. It is equally why **absolute accuracies here do not transfer to a full-Freebase setting**. Every comparison in Section 5 is between systems traversing the identical environment, which is what the comparison is for. None of the numbers should be read against published figures obtained over a different substrate; Section 5.4 shows what such a reading looks like when it is done with the caveats attached.

One limit of the environment concerns expressiveness rather than coverage. Every node carries a single entity label. There is no typed literal, so a date or a number exists only incidentally, as an entity whose name happens to be a numeric string, and the tool API exposes no ordering or aggregation operator. A question whose gold answer is a date, a measured quantity, or a superlative over candidates is therefore unanswerable in this environment by construction. Section 7 reports how much of the residue that class accounts for, and Section 8 carries it as a scope limit.

4.2. Benchmarks, Baselines, and Backbone

We evaluate on WebQSP [8] and ComplexWebQuestions [9], the two standard multi-hop KGQA benchmarks over Freebase. **We sample 400 questions from each** rather than running the full 1,628 and 3,531 test splits. That was a pre-specified fallback, adopted when the full-split cost was projected at roughly three times the budget available for the work. The sample is stratified by hop count at a fixed seed, and the resulting 95% bootstrap intervals are roughly ± 4 to ± 5 points on Hits@1. Section 8 says which of the paper’s claims survive that width and which are reported as directions rather than effects. The two benchmarks differ less in the typical question than in the tail. Exactly half of the WebQSP sample carries a single

gold answer, but its mean is 7.15 gold entities per question and its largest question lists 237. ComplexWebQuestions averages 1.75 with a median of 1. That tail is one reason we report F1 beside Hits@1.

Four baselines span the design space. Each is a paradigm rather than a tuned competitor. A **no-retrieval** control answers from parametric memory alone, establishing what the backbone knows without the graph. **Vector-RAG** retrieves verbalised triples by embedding similarity. **Static GraphRAG** [4] fetches a fixed-radius neighbourhood around the linked topic entities in one shot, reranks it, and answers from the top 30 facts. Two properties of that implementation bound what it licenses, and we state them here rather than leave a reader to infer them from its scores. The radius is *one logical hop*. Mediators are traversed transparently, so a mediator path costs two graph edges while remaining one hop, and a genuine two-hop chain is outside its reach by construction. And expansion retrieves at most 100 edges per topic entity with no ordering criterion, so on a high-degree entity the retained edges are an arbitrary sample rather than the best or the first. At least one topic entity is truncated on 72.5% of the 800 test questions. This baseline therefore licenses one narrow claim: *retrieval fixed before reasoning must commit to a radius in advance, and any fixed radius is wrong for some question in the set*. It does not license a claim about what a two-hop static retriever would score. That experiment was not run.

Think-on-Graph [5] is the agentic comparator. It performs model-guided beam search over relations and entities, and it is the system AGR is measured against throughout. It is a reimplementaion on this environment’s tools, at the published beam width and depth, rather than a quoted number.

That is a stronger comparison than citing the original figures, because graph, model, budget, and scoring rule are held constant and only the algorithm varies. It also means its numbers are not comparable to the published ones, and we do not present them as such. All baselines were debugged on development data and certified before any test question ran, against a pre-specified diagnostic targeting the one failure that would be an implementation defect rather than a paradigm limit: retrieval found the gold answer and the system hedged anyway.

Every system shares one frozen backbone, `gpt-5.4-mini-2026-03-17`, a dated snapshot rather than an alias, at temperature 0.0 with reasoning effort disabled. A check that logs the reasoning-token count on every call read zero across every run in this paper. The systems also share the entity resolver, the question files, the output schema, and *one budget meter with the same 25-call cap*. Both graph-navigating systems seed from the datasets’ annotated topic entities, which removes entity linking as a confound. Holding all of this constant is what licenses the paper’s central inference: a difference between two systems is attributable to architecture, not to model capacity, retrieval corpus, or spend. Two things are deliberately not held constant, and we name them here rather than bury them. The baselines receive a plain answering prompt, while AGR’s drafting prompt carries the grounding provisions that are part of the system under test. And the two navigators consider candidate sets of different widths: 40 relations and 20 neighbours for Think-on-Graph against 300 and 200 for AGR (Table 1). The second is where a reader should look first for a confound, and Section 5.3 reports the measurement that bounds it.

4.3. Metrics and Protocol

Accuracy is reported as **Hits@1** and per-question **F1**. We report F1 because a system can score a clean hit on set-intersection while asserting several wrong entities alongside the right one. A **hedge** is an empty predicted entity set. Hedges score zero on both metrics, and the hedge rate is reported as its own column, so a system buying precision through abstention is visible rather than flattered. Hits, hedges, and wrong answers always partition the whole set. Nothing is dropped from the denominator, including questions the environment provably cannot answer. Matching is exact after Unicode normalisation, case folding, and whitespace stripping, with no fuzzy or substring matching. We measured the obvious worry rather than assumed it away: of AGR’s wrong answers, 1 of 65 on WebQSP and 6 of 99 on ComplexWebQuestions are surface-form near-misses that aggressive normalisation would have scored as hits. The failure census of Section 7 is reading real errors.

Groundedness is measured at two levels. The structural tier is deterministic and applies to every asserted entity in all 4,000 records: an entity is grounded if and only if it exists as a node in the environment and connects to a topic entity within four hops. The semantic tier asks the harder question of whether the entity is supported *as the answer*. For a stratified sample of 60 answers per system per dataset, it retrieves the graph’s own shortest connecting paths and has a language model judge whether any path expresses the relationship the question asks about, under the governing instruction that mere connectedness is not support. That judge was validated against a human annotator on 100 items, reaching 85% agreement

and Cohen’s $\kappa = 0.6995$. That *misses* the 0.70 we pre-specified, by 0.0005, and Section 8 treats the miss as a limitation rather than rounding it away. Because path retrieval takes only the three shortest paths, semantic-tier rates are conservative lower bounds. They are depressed uniformly across systems and read as a between-system comparison rather than as absolute support rates. Efficiency is tokens and model calls, both exact from the shared meter.

The protocol was **pre-specified**. Research questions, metrics, baseline certification thresholds, and ablation conditions were fixed and documented before the test sets were constructed, and no hyperparameter was tuned on test. Four decisions in particular were written down before the measurements they govern: the frozen configuration $\alpha = 0.7$ and $\tau = 0.20$, selected on a disjoint 80-question development set and never revisited; the sample size of 400 questions per dataset, named as the fallback before the cost was known; the baseline certification threshold; and the judge threshold of $\kappa \geq 0.7$. The last was missed, and it is reported as missed. A threshold that counts only when it is met is not a threshold. Nothing was filed with a public registry, so the claim is pre-specification and not pre-registration. Confidence intervals are 95% percentile bootstrap intervals over questions from 10,000 resamples, and paired comparisons use the exact form of McNemar’s test on per-question correctness. Deviations encountered during execution, and there were several, are recorded in the repository rather than narrated here.

5. Results

The same runs are read several times over: what the systems answer correctly, what they spend doing it, where the margin over the agentic

baseline comes from, how the numbers sit against a published system on the same data, how accuracy varies with the number of hops a question requires, and whether the entities the systems assert are supported by the graph they navigated. Where a pass narrows to two systems it says so. Where AGR is not the best of the five, it says that too.

5.1. Accuracy

Table 2 gives the main matrix. AGR leads every accuracy cell on both datasets. It reaches Hits@1 of 0.755 on WebQSP against the agentic baseline’s 0.660, and 0.522 on ComplexWebQuestions against 0.435. The margin over Think-on-Graph is 9.5 and 8.7 points. The per-question F1 gap is wider still, 0.642 against 0.477 and 0.469 against 0.378, because F1 penalises the extra wrong entities that a set-intersection hit forgives. All eight paired comparisons against the four baselines reject at the conventional level. The two against Think-on-Graph, the only comparisons in which both systems navigate the same graph with the same model under the same call budget, rest on 76 questions AGR alone answers against 38 the other way on WebQSP ($p = 4.8 \times 10^{-4}$), and 86 against 51 on ComplexWebQuestions ($p = 3.5 \times 10^{-3}$).

Two features of this table cut against a simple reading. **Raw hits mislead on the static graph baseline, and the correction is itself a finding.** GraphRAG scores 0.340 on WebQSP, below the no-retrieval control’s 0.453. That invites the conclusion that the implementation is broken. Putting the wrong-answer counts beside the hits inverts the picture. The control asserts on 351 of 400 questions and is wrong on 170 of them, an *assertion precision* of 51.6%, where assertion precision is the share of

Table 2: Main results, 400 questions per dataset, one frozen backbone and one 25-call budget for every system. Hedge is the proportion of questions answered with an empty entity set.

System	WebQSP				ComplexWebQuestions			
	Hits@1	F1	Hedge	Calls	Hits@1	F1	Hedge	Calls
no-retrieval	0.453	0.305	12.2%	1.0	0.307	0.279	38.0%	1.0
Vector-RAG	0.568	0.432	27.0%	1.0	0.203	0.168	48.2%	1.0
GraphRAG	0.340	0.262	55.8%	1.0	0.205	0.183	60.8%	1.0
Think-on-Graph	0.660	0.477	18.8%	12.8	0.435	0.378	22.5%	18.2
AGR	0.755	0.642	8.2%	6.2	0.522	0.469	23.0%	8.9

questions a system commits on that it answers correctly. GraphRAG asserts on 177 and is wrong on 41, or 76.8%. The mechanism is the prompt. The retrieval baselines are instructed to answer only from the facts retrieved and to return nothing otherwise, so they abstain when retrieval misses. The control has no such constraint and guesses from memory. On a decade-old benchmark plausibly present in the backbone’s pretraining data, guessing pays. A grounded system scoring below an ungrounded one on raw hits while committing four times fewer falsehoods is the contamination effect the control exists to expose. We report it as a result rather than as an anomaly.

The second is that AGR does not lead on every column. On ComplexWebQuestions its hedge rate of 23.0% is marginally *above* Think-on-Graph’s 22.5%, so it declines to answer slightly more often there. The accuracy lead is not bought by answering more freely, and assertion precision on that dataset makes the point directly. The two systems commit at effectively the same rate, on 308 and 310 of 400 questions. From those near-identical

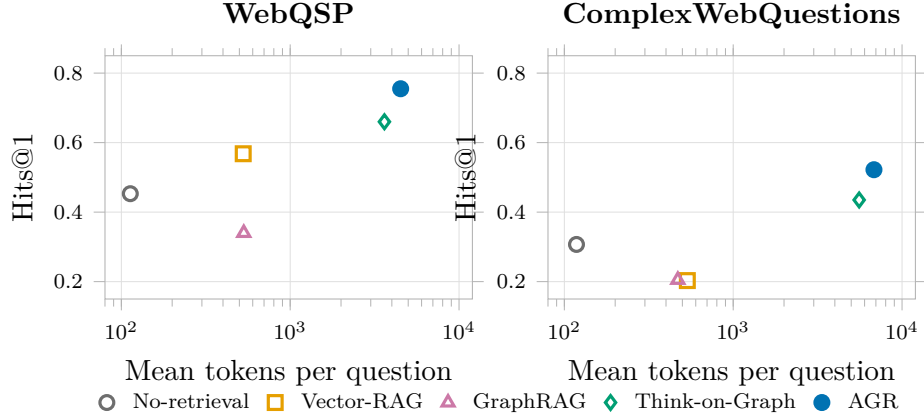


Figure 3: Accuracy against mean tokens per question, both datasets, with cost on a log scale. The single-call baselines cluster at the left. The two navigators sit at the right, with AGR above and to the right of Think-on-Graph on both panels: it buys its accuracy with more tokens per question, not fewer. The halving is in calls, which are not plotted.

commitments, AGR is right 209 times and wrong 99, against 174 and 136 for Think-on-Graph: 67.9% assertion precision against 56.1%. On the deeper benchmark, the system with an explicit plan, a guided scorer, and a claim check finds 35 more answers and asserts 37 fewer falsehoods than beam search does, on the same graph, with the same model, from the same number of commitments.

5.2. Cost

AGR reaches those numbers at roughly half the model calls of the agentic baseline: 6.2 against 12.8 on WebQSP and 8.9 against 18.2 on ComplexWebQuestions, ratios of 0.48 and 0.49. Figure 3 plots accuracy against cost for all five systems.

The call saving is genuine and the token saving is not. AGR spends 4,511

mean tokens per question on WebQSP against Think-on-Graph’s 3,615, and 6,818 against 5,572 on ComplexWebQuestions. It is the *number of round trips* that halves, not the volume of text. Fewer, larger calls is the accurate description. Which cost binds is a deployment question. On the token axis of Figure 3 Think-on-Graph is a frontier point rather than a dominated one, since it buys lower accuracy at a lower token price. On calls it is dominated on both datasets, and calls are what the shared budget meters. Against one-shot retrieval, agency costs roughly an order of magnitude more in tokens. That is the price of the accuracy in Table 2, and it is stated as a price.

AGR never reaches the shared call cap. Across all 800 test questions it hit the 25-call limit zero times. That is not the same as running unbounded: its depth cap of Table 1 refuses a deeper expansion on 25.6% of test questions. What the call budget establishes is the narrower fact that of two systems run under one shared cap, that cap truncates only one of them. The next subsection shows what that truncation does to the comparison.

5.3. *Where the Margin Comes From*

The more informative result is *where* the accuracy margin comes from, and it is not where a reader would assume. Beam search costs width times depth, so the shared 25-call cap binds on Think-on-Graph far more often than on AGR. Splitting the questions by whether the cap cut Think-on-Graph off before it finished separates two things the aggregate confounds: how good its search is, and whether it can afford to finish that search. Table 3 gives the split, and it reverses the comparison on one side.

On the questions where the budget lets Think-on-Graph run to completion, it is *ahead* of AGR: 0.852 against 0.788 on WebQSP, and 0.630 against

Table 3: AGR against Think-on-Graph, split on whether the shared call cap truncated the baseline. Read from the per-record `budget_exhausted` trace flag.

Subset	n	Think-on-Graph Hits@1	AGR Hits@1
<i>WebQSP</i>			
Think-on-Graph finished	283	0.852	0.788
Think-on-Graph clipped	117	0.197	0.675
<i>ComplexWebQuestions</i>			
Think-on-Graph finished	224	0.630	0.607
Think-on-Graph clipped	176	0.188	0.415

0.607 on ComplexWebQuestions. The entire aggregate margin comes from the 29% and 44% of questions on which the cap truncates it. There its accuracy collapses to 0.197 and 0.188, while AGR still reaches 0.675 and 0.415.

The correct reading of this is affordability, not resolution quality. Exhaustive beam search finds answers well when it is allowed to finish. Guided search finds them almost as well and finishes within a budget the beam search cannot. On a fixed spend the second is the better architecture. The first would win if spend were unconstrained. That is a claim about operating regimes rather than about reasoning power, and it is narrower than the aggregate table alone would suggest. The cap of 25 calls is itself a choice, and a larger one would move the split. The clip rates are reported so a reader can see how far.

One asymmetry bounds how far that reading extends, and it is the candidate-width difference named in Section 4.2. Both navigators call the same tools, but they truncate what those tools return at different

widths. Think-on-Graph keeps the first 40 relations per entity and the first 20 neighbours per relation, the pruning widths of the reimplemented algorithm, against AGR’s 300 and 200. The relation list is returned sorted by *descending* fanout, so a binding cut discards the low-fanout tail. That tail is where the specific relation answering a specific question tends to sit, since a high-fanout relation is rarely the edge that resolves a particular entity. Over the committed tool logs the 40-relation cut binds on 31.6% of the 1,651 entities Think-on-Graph expanded and the 20-neighbour cut on 32.8% of its 7,992 neighbour calls, against 1 of AGR’s 3,097 expanded entities and 3.3% of its neighbour calls.

The affordability conclusion survives this, because a thinner candidate pool cannot explain why a system runs *out of calls*. What it bounds is the other half of the split. The 0.852 and 0.630 the baseline reaches on the questions it finishes were measured while it searched a narrower pool than AGR. They are therefore a **lower bound** on what an equal-width reimplementation would reach, not an estimate of it. And since the baseline is already ahead of AGR on those questions, equalising the widths would be expected to widen that lead rather than close it. Beam width and depth follow the original paper. The candidate widths beneath them do not, and equalising them is the first change we would make in a follow-up. That re-run would not vary one thing, though. Wider candidate sets make each pruning call dearer, so equal widths would also raise the baseline’s clip rate. That moves the boundary between the two halves of Table 3 at the same time as it changes what the search resolves within each. The two effects have to be read apart, which means reporting that split again rather than an aggregate

in which they cancel or compound invisibly.

The breadth of AGR’s answers is checked rather than assumed, because a metric that scores a hit on any match rewards naming more entities. On the WebQSP questions it answers, AGR names 3.37 entities against Think-on-Graph’s 2.56, and its entity precision over those same answered questions is 0.760 against 0.703. On ComplexWebQuestions the figures are 1.54 against 1.40 entities and 0.629 against 0.520 precision. If AGR bought breadth without accuracy, precision would fall as the entity count rose. It does not fall.

5.4. *Positioning Against Published RoG Results*

Two facts in this paper invite a comparison that the controlled protocol above cannot make. The environment of Section 4.1 is built from the per-question subgraphs that the RoG distribution publishes, and Section 2.2 surveys RoG [7] as the closest published system in the path-retrieval family. A reader who notices both will ask how the two systems compare. The aggregate answer does not favour this work, and Table 4 gives it before the qualifications rather than after them.

RoG leads on all four figures, and the margin is not sampling noise. AGR’s 95% intervals on Hits@1 are [71.3, 79.7] on WebQSP and [47.5, 57.0] on ComplexWebQuestions. RoG’s 85.7 and 62.6 fall outside both. Three differences bound what the table establishes. The first is decisive and the other two are ordinary. **RoG is trained on these benchmarks; AGR is trained on nothing.** RoG’s backbone is instruction-tuned on the training splits of both benchmarks, which hold 2,826 and 27,639 questions in the distribution used here. AGR’s backbone is frozen and prompted, and it has

Table 4: AGR against RoG’s published figures, in points. RoG’s row is as published in [7], measured over the full test splits. AGR’s row is Table 2, measured over the certified 400-question samples, which are stratified draws from those same splits. The two rows are computed over nested populations under different protocols; the text says what that permits.

System	WebQSP		ComplexWebQuestions	
	Hits@1	F1	Hits@1	F1
RoG (published)	85.7	70.8	62.6	56.2
AGR (this work)	75.5	64.2	52.2	46.9

seen no question from either benchmark before evaluation. The comparison is between a supervised system and a zero-shot one, which is a difference in kind rather than in degree. **The backbones differ.** RoG fine-tunes LLaMA2-Chat-7B. Section 4.2 holds the backbone constant precisely because it accounts for much of the spread in cross-paper tables, and it cannot be held constant against a system whose contribution *is* its fine-tuned weights. **The populations differ.** RoG reports over the full test splits. Our figures are over stratified 400-question samples of the same splits, which estimate the same quantity, with the intervals above as their uncertainty. RoG publishes no interval, so the gap can be bounded from one side only.

What the comparison does settle is worth stating positively. It fixes AGR’s place in the published landscape: a training-free system reaching 75.5 and 52.2 against a supervised system’s 85.7 and 62.6 on the same data, with no in-domain examples. And it locates this paper’s contribution somewhere other than the accuracy column. RoG’s answers are read off retrieved paths, so they are faithful by construction, and the pre-generation

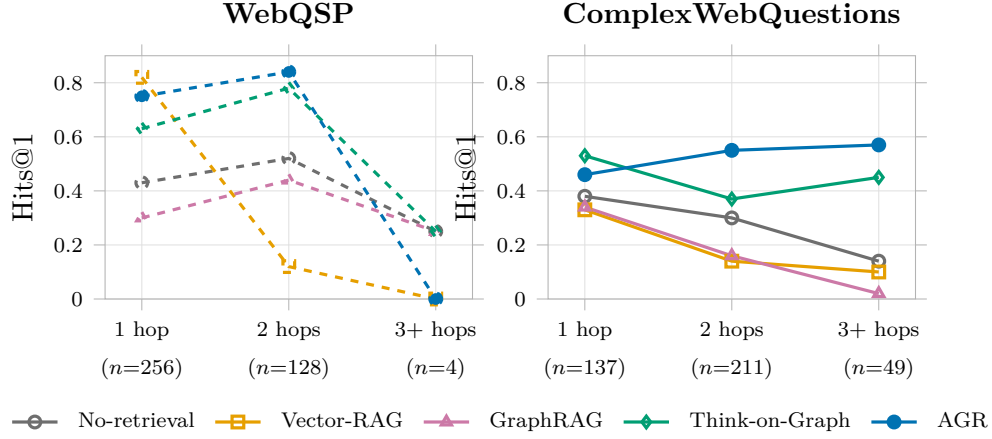


Figure 4: Hits@1 by hop stratum. On ComplexWebQuestions (right) AGR is the only system whose accuracy rises monotonically with hop count. The WebQSP three-hop stratum holds four questions and no weight should be placed on it.

verification question never arises for them. The price is a plan generated once, before retrieval, with no mechanism to revise it when it fails to instantiate. AGR’s claims are the opposite shape: generated freely and then checked against the graph the agent walked, which is what makes an unsupported assertion detectable at all. Neither the component-level ablation of Section 6 nor the failure census of Section 7 has a counterpart in RoG’s evaluation, and closing the gap in Table 4 would settle neither.

5.5. Accuracy by Hop Depth

Figure 4 stratifies by the number of hops separating a topic entity from the gold answer.

On ComplexWebQuestions, AGR is the only system whose accuracy *rises* with hop count: 0.46 at one hop, 0.55 at two, 0.57 at three or more. Every

other system ends below where it started. That phrasing is deliberate, and the weaker alternative is the true one. Think-on-Graph does not decay monotonically. It falls from 0.53 to 0.37 and then partly recovers to 0.45, which is a different shape from a steady decline, and we report it as such. Think-on-Graph also beats AGR on that dataset’s one-hop stratum, 0.53 against 0.46. Section 6.1 identifies the mechanism: decomposition costs accuracy on questions that do not need decomposing, and that stratum is full of them.

The static graph baseline’s decay across these strata is deliberately *not* offered as evidence for the paradigm, because its retrieval radius confounds it. A one-hop retriever cannot contain a two-hop chain, so part of that decline measures the radius rather than static retrieval as such. The evidence that it is a radius effect is in the strata themselves. GraphRAG scores 0.44 on WebQSP’s two-hop stratum, where those questions are largely mediator paths a one-hop expansion does reach, against 0.16 on the ComplexWeb-Questions two-hop stratum, where the chains are genuine compositions it cannot reach at all. A retriever failing uniformly on depth would not show that gap. Where the per-stratum argument needs a static comparator that is not radius-limited, it uses Vector-RAG, whose one-triple context cannot represent a chain at any radius.

A rising curve is the signature the architecture predicts. Decomposition converts one hard question into several easy ones, so the questions that punish single-shot retrieval hardest are the ones an agent that re-plans between hops should handle best. The WebQSP panel carries no comparable evidence. Its three-hop stratum contains four questions, which is why the claim is scoped

to one dataset.

5.6. Groundedness

Across both datasets AGR asserted 1,709 entities, and **zero of them were ungrounded**. Every entity it named exists in the environment and is reachable from a topic entity within the hop cap. The parametric control asserted 1,001 entities, of which 22.1% failed that test.

This result belongs to graph navigation, not to the verification layer, and the distinction matters. Think-on-Graph also reaches exactly zero ungrounded assertions, on 1,265 entities, without any claim-checking component whatsoever. Whatever the verification layer contributes, it is not this number. What the comparison establishes is that an agent which can only name entities it retrieved cannot fabricate one, and that this property follows from the retrieval discipline rather than from any check applied afterwards. That conclusion costs this paper its most quotable claim for the component it contributes. It is also the conclusion the measurement supports.

The semantic tier asks the harder question of whether an asserted entity is supported *as the answer*, and it separates the systems far less. Table 5 gives every cell rather than a selection, because the ordering differs by dataset and any subset of it misleads.

Every system lands in a 36.7–66.7% band. AGR is highest on WebQSP at 66.7%. **On ComplexWebQuestions it is second**, 5.0 points ahead of the agentic baseline but 1.7 behind Vector-RAG’s 50.0%. Its margin over Think-on-Graph is 5.0 points on both datasets. Its lead over *every* system is a WebQSP result only, and we say so rather than quote the two comparators

Table 5: Semantic-tier support rate: the share of sampled assertions a judge found supported *as the answer* by the graph’s own connecting paths. $n = 60$ per cell, so the 95% binomial interval has a half-width of roughly 12 to 13 points. These are conservative lower bounds by construction (Section 4.3).

System	WebQSP	CWQ
no-retrieval	63.3%	36.7%
Vector-RAG	61.7%	50.0%
GraphRAG	55.0%	36.7%
Think-on-Graph	61.7%	43.3%
AGR	66.7%	48.3%

it happens to beat.

Neither gap survives the sample. At $n = 60$ per cell a 5-point difference sits well inside the interval, as does Vector-RAG’s 1.7, and no conclusion in this paper rests on either. What the tier establishes is the level rather than the ranking, and the level is the more useful finding: **zero structural hallucination coexists with between a third and nearly two-thirds of assertions failing a strict semantic-support test**, for every system, including the two that are structurally perfect. Semantic support is hard for the paradigm, not for one system within it.

6. Component-Level Attribution

Agentic systems are assemblies, and the literature reports them whole. This section takes AGR apart. Four conditions each remove exactly one component from the unmodified system and re-run the identical questions: no planner, no backtracking, no claim verification, and embedding-only scoring

Table 6: Component ablations. $\Delta F1$ is ablated minus reference: positive means the system got better without the component. p from McNemar’s test on paired per-question correctness.

Removed	WebQSP ($n = 200$)			CWQ ($n = 198$)		
	$\Delta F1$	p	Tokens	$\Delta F1$	p	Tokens
Planner	+0.083	0.006	−31%	−0.047	0.088	−21%
Backtracking	−0.013	0.727	−7%	0.000	1.000	−18%
Verification	+0.004	1.000	−2%	−0.009	1.000	−6%
Model scoring	−0.016	0.664	−25%	−0.008	0.481	−31%

($\alpha = 1$, disabling the model term in the edge scorer). Each condition is compared against the reference on the same half-split, $n = 200$ on WebQSP and $n = 198$ on ComplexWebQuestions, with McNemar’s test on the paired per-question outcomes. The reference row is the unmodified system restricted to the same questions, read from the main-run records rather than re-run, so it is provably the system that produced Table 2.

Throughout this section a delta is ablated minus reference, so a positive number means removing the component *improved* the metric. Table 6 gives all four. The p -values are reported as computed. A correction for multiple comparisons was not among the pre-specified decisions, and applying one after seeing the results would be a choice made on the results, so the four tests are read as descriptive rather than as a family-wise claim.

6.1. When Planning Hurts

One component produced a detectable effect, and it produced it in the wrong direction. **Removing the planner improves WebQSP F1 by**

0.083 ($p = 0.006$) while cutting token use by 31%. Twenty-one questions were answered correctly only without the planner, against six correct only with it. The disagreement is not marginal, and the cheaper system is the more accurate one.

The mechanism is over-decomposition. WebQSP is predominantly a single-hop benchmark: 256 of its 400 questions need one hop. Asked to decompose a question that has no compositional structure, the planner manufactures sub-objectives that are not really separable. The explorer then scores candidate edges against a fragment of the question rather than against the question, and it discards exactly the context that would have identified the right relation. The planner prompt carries a single-hop exemplar precisely to show that not decomposing is a valid output, and that exemplar is evidently not enough. The per-stratum breakdown locates the damage. On the WebQSP half-split, removing the planner raises one-hop Hits@1 from 0.75 to 0.85 and two-hop Hits@1 from 0.84 to 0.89, so decomposition hurts simple and two-hop questions alike on that benchmark.

On ComplexWebQuestions the direction reverses, as the mechanism predicts. That benchmark is genuinely compositional, only 137 of 400 questions are single-hop, and removing the planner costs 0.047 F1. **That arm does not reach significance** ($p = 0.088$), and we report it as a direction rather than an effect. We do not describe it as marginally significant, because at this sample size that phrase would assert more than the data carries. The strata show a coherent mechanism rather than noise. Without the planner, one-hop Hits@1 *improves* from 0.49 to 0.54, two-hop collapses from 0.48 to 0.34, and three-hop-plus moves from 0.62 to 0.54. That is exactly the pattern

in which decomposition helps genuine composition and hurts questions that do not need it.

Pooling the two datasets would average a significant improvement against a non-significant harm and report approximately nothing. The asymmetry *is* the result. Decomposition is not a general-purpose benefit. It is a mechanism whose value depends on whether the question has compositional structure to find. The actionable consequence follows directly, and it is the recommendation we would most like to see adopted: **gate decomposition on question structure** rather than applying it unconditionally, predicting the number of hops from the question and invoking the planner only above a threshold. On these benchmarks that policy would capture the WebQSP improvement and the ComplexWebQuestions capability at once.

6.2. Three Components with No Detected Effect

The remaining three conditions moved nothing detectable, and we report them at the same length as the one that did. A field that reports only what moved cannot attribute anything.

Backtracking. Removing it changes WebQSP F1 by -0.013 ($p = 0.727$) and ComplexWebQuestions F1 by 0.000 ($p = 1.000$), while saving 7% and 18% of tokens. The mechanism fires rarely. The reference condition records 38 and 108 backtracks over the 398 paired questions, and on those same runs the meter refuses a further attempt on 28 of them, 7.0%. A component this seldom exercised cannot show a large effect. One implementation detail bounds the null further, and we record it rather than smooth it over. The backtracker bans the edges of the most recent explorer pass but restores the highest-scoring snapshot, which is often older, so everything expanded in

between stays unbanned. Over the main runs, 53 explorer passes re-expanded an anchor-and-relation set the same question had already expanded, and at least 65 of the 262 backtracks restored a depth other than the most recent snapshot’s. The null therefore describes backtracking as implemented, not backtracking as specified. The accurate summary is that the search rarely needs to undo a decision, not that undoing is worthless.

Model scoring. Setting $\alpha = 1$ removes the language model from edge selection entirely, leaving pure embedding similarity. F1 falls by 0.016 and 0.008, neither significant, while tokens drop 25% and 31%. The model term is the single most expensive part of the navigation loop, and this experiment cannot establish that it earns its cost. One confound is recorded rather than smoothed over. At $\alpha = 1$ the scorer takes an early return that leaves the backtracking trigger’s signal maximum computed over a smaller candidate set, so this condition’s elevated backtrack count is not attributable to the scoring change alone.

Claim verification. Removing the layer this paper contributes changes WebQSP F1 by +0.004 and ComplexWebQuestions F1 by -0.009 , $p = 1.000$ on both. Across the half-splits — 200 questions on WebQSP and 198 on ComplexWebQuestions — the two conditions disagreed on exactly one question each. **Claim verification does not measurably change how often AGR is right.**

The result is expected rather than disappointing, for the reason Section 3.2 gave before any of these numbers appeared. On most questions the layer is a pass-through certifying a draft that was already grounded. On test it asks for a repair on 52 of 800 questions, exploration actually resumes on

28, and 13 end grounded that did not start so. A mechanism that alters the output on a few per cent of questions cannot move an aggregate accuracy metric. An ablation is the right instrument for establishing that, rather than assuming it either way.

What the layer delivers is therefore not an accuracy gain but the output contract: an answer paired with the traversed triples that support it, and an entity list that is a deterministic function of the verdicts rather than a second generation. That is an auditability claim. It is what Section 5.6 shows the layer cannot claim credit for on groundedness, and it is the claim the measurements support.

6.3. Statistical Power

These are nulls at low power, not demonstrations of absence, and the arithmetic is worth stating exactly. McNemar’s exact test sees only the *discordant* pairs, the questions on which the two conditions disagree, and the null conditions produced very few of them.

Removing claim verification changed the outcome on exactly one question per dataset. With a single discordant pair no split whatsoever can reach significance. So that row is not a test that failed to reject. It is the two conditions agreeing on 396 of the 398 paired questions, which is a statement about how rarely the layer alters an answer rather than about statistical power. Backtracking produced 8 and 11 discordant pairs, and model scoring 21 and 18. At those counts the smallest imbalance the exact test could have called significant is 8 and 9 questions for backtracking, meaning all eight would have had to fall the same way, and 11 and 10 for model scoring, which is between 4.0 and 5.5 points of accuracy. Differences below that were

undetectable at any power, whatever their true size.

For calibration: detecting a true 2:1 discordance ratio at 80% power requires roughly 72 discordant pairs, and the largest of these conditions produced 21. At that count the smallest ratio detectable at 80% power is nearer 4:1. A true 2:1 effect would have been caught about a quarter of the time. **Three of four components show no effect at this power;** none is shown to have no effect.

7. Error Analysis

Every one of AGR’s 259 remaining failures, 86 on WebQSP and 173 on ComplexWebQuestions, was read and labelled by hand against its full execution trace. The census is a population, not a sample: every question the system did not answer correctly was read, so the counts below are counts rather than estimates. The reading protocol was fixed before it began. For each trace, find where the trajectory *first* left the correct path, and label that point rather than the last visible symptom. Wrong answers and hedges are kept separate throughout, because the two are not the same kind of failure. A wrong answer is a reasoning error. A hedge is usually a coverage gap that never produced a committal claim. Figure 5 gives the census.

The bulk of the distribution is unsurprising. Choosing the wrong relation to follow is the base rate of graph navigation, and it is the largest category overall. Composite claims, in which an unverifiable component drags down a verifiable one, are nearly absent on WebQSP and the largest category on ComplexWebQuestions, because that benchmark’s template-generated questions carry conjunctions and constraints that WebQSP’s search-log ques-

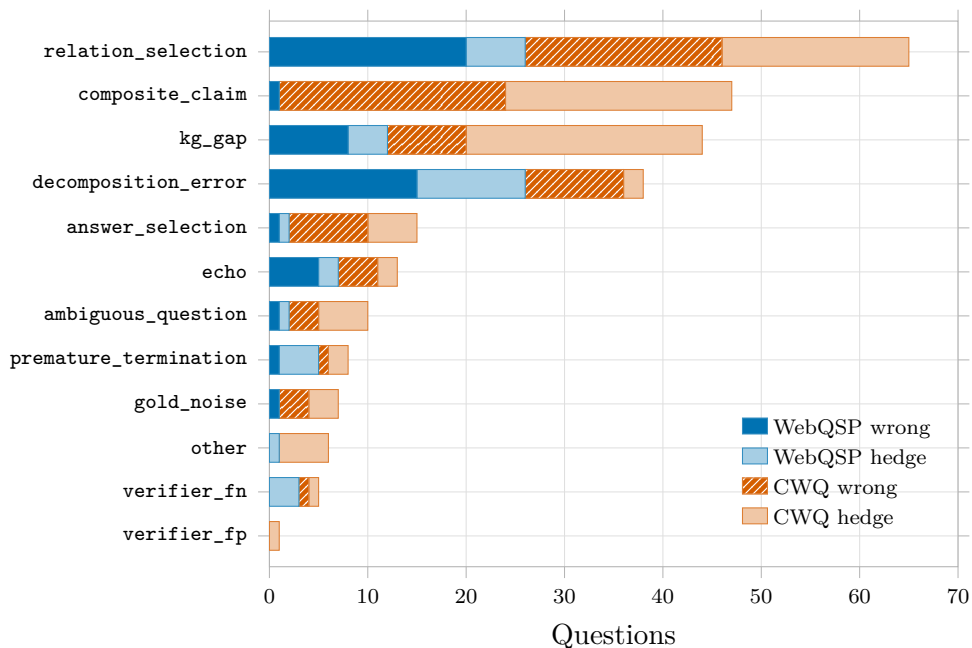


Figure 5: Hand-read census of all 259 residual failures, by category, with wrong answers and hedges kept separate. Relation selection and composite claims dominate. The two categories this section discusses are not the largest, but they are the two whose lessons transfer beyond this system.

tions rarely do. The same construction explains the third-largest category. Questions whose answer is a date, a number, or a superlative fall into the class Section 4.1 named as unanswerable in this environment, and on Complex-WebQuestions the system hedges on them three times as often as it asserts something wrong, 24 against 8. That is the behaviour the drafting prompt’s grounding provisions were written to produce, visible in the one place where the environment guarantees the system cannot succeed. Two smaller categories are reported in full instead, because they generalise past this architecture.

7.1. The Echo Attractor

The most instructive failure mode is one that every grounding check in this paper passes. The system resolves a real entity, reached through a real edge, and states a fact that is genuinely true. It is not what was asked. We name this the **echo attractor**: a true, well-grounded entity sitting one hop from the answer in the graph’s structure.

It takes two shapes. *Topic confusion* picks a neighbour in a tight naming cluster. Asked who plays Lex Luthor on *Smallville*, the system answers that John Glover plays Lionel Luthor, a different character in the same fictional family, with three further Luthor variants surfacing in the trace before the run gives up. *Granularity* picks the right entity family at the wrong level. Asked for a country, the system answers “Kingdom of Denmark” where the gold answer is “Denmark”, a genuinely distinct node, the formal sovereign-state entity rather than the common-name one.

What makes this a mechanism rather than a residue category is that nothing went wrong in the parts of the system built to prevent things going wrong. The right edge exists, is retrieved, and is walked. The entity returned is verifiably present in the environment. Every structural check certifies it. The failure is entirely in *which* true fact gets surfaced, which is architecturally distinct both from following a wrong edge and from the fact being absent. A claim-level check against traversed adjacency cannot catch it by construction. The claim is true.

The consequence for evaluation methodology is larger than the consequence for this system. The attractor is a property of the graph’s neighbourhood structure rather than of any one search policy. So architec-

turally unrelated systems converge on the *same* wrong entity for the *same* questions. Any protocol that treats systems as independent observers therefore cannot see it. Consensus filtering, ensembling, and majority-vote gold repair all take agreement as evidence of correctness, and here agreement is evidence of a shared structural bias instead. Such a protocol will confidently ratify the wrong answer in exactly the cases it was introduced to catch. We observed this directly in the pass described next. Five systems agreeing on one answer turned out far more often to be five systems echoing than to be a bad label.

The echo attractor accounts for 13 of the 259 failures. The rate is small and the sample does not support a precise one. The contribution is the named mechanism and its methodological consequence, not its frequency at this n .

7.2. *Benchmark Label Defects*

The census required a second pass, because a failure against a wrong gold label is not a failure of the system. A consensus pre-pass flagged every question where at least three of the five systems agreed on an answer the benchmark scored wrong. Each flagged question was then adjudicated individually against the graph and the question text.

The two counts must be kept apart. The pre-pass **flagged** 58 WebQSP and 47 ComplexWebQuestions questions. Adjudication **confirmed** a genuine label defect in 22 and 19 of them, which is 5.5% and 4.8% of each sample. Confirmed defects split into two kinds: gold answers that are simply wrong (12 and 17), and questions whose phrasing admits several defensible answers of which the benchmark lists one (10 and 2). All 41 were removed before the

census began, as benchmark artifacts rather than system failures.

The hand-read then found 17 more that the pre-pass had missed — 3 on WebQSP and 14 on ComplexWebQuestions — and filed them under its own gold-noise and ambiguous-question labels. One question belongs to both sets, so 57 **distinct questions** across the two benchmarks turned out to need correcting. That the census recovered defects a dedicated consensus pass had missed is itself an argument for reading failures individually rather than screening them in aggregate.

The gap between flagged and confirmed is not adjudication noise. It is the echo attractor, measured. The pass flagged 105 questions and adjudication confirmed 41. So roughly three-fifths of what it flagged was not a bad label at all but several systems converging on the same structurally-adjacent wrong entity, which is precisely the failure mode the previous subsection predicts a consensus protocol will misclassify. Had we accepted the pre-pass without per-item adjudication, we would have “corrected” the benchmark toward the systems’ shared bias and reported an inflated accuracy for every system, including our own.

The accuracy tables are not rescored. Section 5 reports every system exactly as measured over the full samples, for two reasons. Rescoring against labels we adjudicated ourselves would substitute one contestable ground truth for another. And the defects cut both ways. A broken label can just as easily hand a system a hit as deny it one, and 10 of the 22 WebQSP exclusions and 4 of the 19 on ComplexWebQuestions are questions AGR scored as hits against a label now judged defective.

That symmetry also bounds how much the exclusions could matter, which

is worth stating because the defect rate on its own invites a much larger inference. Recomputing every cell of Table 2 with the adjudicated questions removed moves each system by between +0.001 and +0.020 Hits@1 and leaves the ranking on both datasets unchanged. That shift is an order of magnitude below the 5.5% and 4.8% defect rates, because removing a question the system got right subtracts from the numerator as well as the denominator. The exclusions therefore do not threaten any comparison this paper draws. What they do establish is that a non-trivial share of both benchmarks is mislabelled, which is a fact about the benchmarks rather than a correction to be applied to them.

8. Discussion and Limitations

The limits of this study bound its claims, and we state all of them rather than the flattering subset. The first group bounds the contribution itself, the second bounds the comparisons, and the third bounds the measurements.

8.1. *What Bounds the Contribution*

Wrongful acceptance is unmeasured, and it is the most serious gap in this work. The verifier logs the claims it rejects and not the ones it accepts. So a claim certified on an edge unrelated to the relation it asserts leaves no trace. The exposure is real rather than hypothetical, because both structural routes ignore the relation and the direction. Any edge between two entities certifies any claimed relation between them, so a claim that X is Y’s mother is accepted on an edge recording that X is Y’s child. We therefore report one polarity at census scale and the other anecdotally, and that asymmetry is a property of the logging decision rather than of the

verifier. There is no way, from the committed record, to know how many wrongful acceptances sit inside the 761 questions that ended grounded.

The exposure is best stated in the claim unit, where its scale is visible. On test the verifier decomposed 2,110 claims and accepted 2,008 of them. The graph-wide adjacency probe accounts for 39 of those acceptances. The rest were made either by traversed adjacency, which is equally relation-blind, or by the entailment check, which is not, and the record does not say which. So the share of accepted claims certified without any test of the asserted relation lies somewhere in $[39, 2,008]$. That interval is two orders of magnitude wide, and we report it as one rather than choose a point inside it.

The output contract cannot be audited from the run records. This paper offers, as its contribution, an answer paired with the traversed triples supporting it. The pairing does hold inside the run: the answerer receives the verifier’s triple list and emits against it. But the logger writes the *count* of supporting triples and discards the list, so no committed artifact in this work contains a single supporting triple. A reader can confirm that these answers came from a system that tracked its evidence. They cannot inspect that evidence. Persisting accepted claims with their matching triples is one logging change, and it would repair this limitation and the previous one together.

Structural grounding is not answer adequacy. Both navigating systems assert zero ungrounded entities. That says their answers are made of the graph’s own material. It does not say they answer the question. The echo attractor of Section 7.1 is the counter-example: a wrong answer that passes every check this paper applies. The layer this paper contributes offers

no protection against it.

8.2. What Bounds the Comparisons

Sample size. Every result rests on 400 questions per dataset rather than the full splits, giving 95% bootstrap intervals of roughly ± 4 to ± 5 points on Hits@1. The main accuracy margins clear that width: 9.5 points over the agentic baseline on WebQSP and 8.7 on ComplexWebQuestions. So does the planner effect, which is a paired test on per-question outcomes rather than a comparison of two intervals. The groundedness contrast, at zero against 22.1%, is not close. What the sample does not support is fine distinctions. The semantic-tier gaps of Section 5.6 and the hedge-rate difference on ComplexWebQuestions are within noise, and we report them as such.

An identical environment is not identical access to it. The agentic baseline truncates candidate sets at 40 relations and 20 neighbours against AGR’s 300 and 200. That cut binds on about a third of its expansions and effectively none of ours (Section 5.3). The comparison holds the graph constant and does not hold the view of it constant, which is why the baseline’s figures on the questions it finishes are a lower bound rather than an estimate.

The static graph baseline is radius-bounded. Its one-logical-hop expansion cannot contain a two-hop chain, so its decay across hop strata measures the radius as much as the paradigm, and Section 5.5 declines to offer it as evidence for either. Its unordered 100-edge fanout cap compounds this.

One backbone. All five systems run on `gpt-5.4-mini`, frozen and closed. This paper establishes how these architectures compare *on one backbone*, not

how the comparison scales across model capacities.

Nondeterminism. Temperature-zero decoding against a hosted API is greedy, not deterministic, and measured trajectory stability is about 67%. Each system was run once. No seeds were averaged, and a reader should not assume they were. The intervals and paired tests resample questions, so they bound the sampling variance of the question draw and nothing else; each system’s answer set enters them as fixed. Two things make a rank reversal from decoding noise alone unlikely: the defended margins are large relative to their intervals, and the paired tests are computed per question. But unlikely is the strongest statement available. The repeated matrix that would bound the variance was affordable at the observed cost and was not run, so this is a gap in the design rather than a budget constraint.

Ablation power. The four ablations run on half-splits of $n = 200$ and $n = 198$, and the paired test sees only the questions on which two conditions disagree, as few as one for claim verification. Three of four components show *no effect detected at this power*, which is not the same as no effect. Section 6.3 gives the smallest difference each condition could have detected, between 4.0 and 5.5 accuracy points where a test was possible at all.

8.3. What Bounds the Measurements

The environment is friendlier than the knowledge base it stands in for. Per-question subgraphs with a 97%-plus answer-reachability ceiling are a more forgiving substrate than full Freebase. Between-system comparisons hold, because all five systems navigate the identical graph. Absolute accuracies do not transfer, and Section 5.4 shows the size of the gap to a published system that was also trained on the benchmarks.

Topic entities are given, not linked. Both graph-navigating systems seed from the datasets’ annotated topic entities. That isolates navigation from entity linking, which is the intent. But it means every reported accuracy assumes a linking step a deployed system would have to perform, and which would introduce errors of its own.

Entities are keyed by name, so homonyms merge. Distinct real-world entities sharing a surface form collapse to one node. That can manufacture a path between things not actually connected, letting a wrong answer verify as grounded, and it can equally destroy a distinction a question depends on. Neither direction is measured. Structural groundedness is therefore a claim about this graph’s node identity rather than about the world.

The measured accuracy has an unmeasured floor. An extraction defect converts correct reasoning into a wrong entity list on one specific question shape. On “list the professions” and “what did X do” questions the drafted text names every gold value and the scored entity list collapses to the sentence’s subject. Accuracy on that shape is a lower bound rather than an estimate.

The judge missed its own bar, and the adjudication was single-annotator. Agreement between the semantic-tier judge and a human annotator reached $\kappa = 0.6995$ against a pre-specified 0.70. That is a miss, by 0.0005. It is why those numbers are reported as corroborating rather than decisive, and why no conclusion rests on them alone. The judge also runs on the same backbone as the systems it judges. Separately, the benchmark-defect exclusions rest on single-annotator judgements made by an author with the

consensus signal visible. One development-set relabelling was also made after its outcome was known. It is flagged in place, and no aggregate was rescored.

Scope. English factoid questions over a static graph, with no temporal facts, no graph construction, and an offline prototype. The environment carries no typed literals and the tool API no ordering operator, so questions answered by a date, a quantity, or a superlative are out of reach by construction (Section 4.1). Latency figures are a cost measure, not a production-readiness claim.

8.4. Outlook

The most actionable finding points forward rather than back. If decomposition helps only when a question has compositional structure, improving accuracy by 0.083 F1 where that structure is absent and costing 0.047 where it is present, then the planner should not run unconditionally. A hop-count predictor gating the planner would capture both arms of that asymmetry, and it can be evaluated with the ablation instrument used here. More generally, the result suggests that agentic components should be treated as conditionally rather than universally beneficial, and that the condition is worth measuring. The single highest-value change to this system is smaller still. Persist accepted claims with the triples that certified them. That converts the two limitations opening this section from acknowledged gaps into measurable quantities.

9. Conclusion

We presented AGR, a knowledge-graph question-answering framework built as an explicit state machine over a constrained graph tool API, whose

answers are paired with the traversed triples that support them. Against four baselines sharing one frozen backbone and one call budget, it leads every accuracy cell on both benchmarks at roughly half the model calls of the strongest of them, and it is the only system whose accuracy rises with hop count on the compositional benchmark. Its advantage is located rather than asserted. On the questions where the shared budget lets the agentic baseline finish, that baseline is slightly ahead. The entire aggregate margin comes from the questions where the budget truncates it. The finding is about affordability under a fixed spend, and we report it as such. Against the published figures of a system fine-tuned on these benchmarks, AGR trails on every number, and we report that too.

The result we consider most useful is the one that did not go our way. Agentic systems are assemblies, and the field reports them whole. We took this one apart, and three of its four components could not be shown to earn their cost. That includes the claim-verification layer this work set out to contribute, which does not measurably change how often the system is right. It changes how often the system is right for a reason it can exhibit, which is an auditability claim and not an accuracy one. The fourth component was actively harmful on the shallower benchmark. Removing the planner improved accuracy and cut token use, because decomposing a question that has no compositional structure discards the context that would have answered it.

The shape of the gap that remains matters as much as the margin. A system that traverses a real graph and copies its answers out of real triples can be made to assert nothing ungrounded. That problem is solved, and the navigation paradigm solves it, not any checking layer built on top. What

remains is an answer that is grounded, connected, and true, and is still not the answer to the question that was asked. Fact verification against a knowledge graph turns out to be the easier half of the problem. Relevance verification is the half still open.

The attribution result is a claim about how this field measures rather than about this architecture. A component adopted because the assembly containing it performed well may be contributing nothing, or less than nothing, and the only way to know is to remove it and test. We would rather report three nulls and a harmful component honestly than a system that works for reasons nobody has checked.

CRedit authorship contribution statement

Md. Sakif Khan: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Sadia Sharmin:** Conceptualization, Methodology, Supervision, Project administration, Writing – review & editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare no competing financial interests or personal relationships that could have influenced the work reported here.

Data availability

Code, prompts, per-question run records, and the analysis scripts that produce every figure and table in this paper are available at <https://github.com/sakif-khan/agent-ic-graph-reasoning> under the MIT licence. The question sets are the public WebQSP [8] and ComplexWebQuestions [9] releases, and both they and the Freebase-derived [3] subgraphs arrive through the public Reasoning-on-Graphs distribution [7] (`rmanluo/RoG-webqsp` and `rmanluo/RoG-cwq`), used unmodified. The knowledge environment is rebuilt from that distribution by the scripts in the repository rather than redistributed.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) to draft and revise the prose of this manuscript from the completed thesis on which it is based, and to write the scripts that check the values quoted in the text against the study’s recorded results. The study design, the experiments, the analysis, and the findings reported here are the authors’ own and predate the preparation of this manuscript. After using this tool the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

- [1] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, T. Liu, A survey on hallucination in large

- language models: Principles, taxonomy, challenges, and open questions, *ACM Transactions on Information Systems* 43 (2) (2025) 1–55.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: *Advances in Neural Information Processing Systems*, Vol. 33, 2020, pp. 9459–9474.
- [3] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, J. Taylor, Freebase: A collaboratively created graph database for structuring human knowledge, in: *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, 2008, pp. 1247–1250.
- [4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, J. Larson, From local to global: A graph RAG approach to query-focused summarization, *arXiv preprint arXiv:2404.16130* (2024).
- [5] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, L. M. Ni, H.-Y. Shum, J. Guo, Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph, in: *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2307.07697.
- [6] L. Chen, P. Tong, Z. Jin, Y. Sun, J. Ye, H. Xiong, Plan-on-graph: Self-correcting adaptive planning of large language model on knowledge graphs, in: *Advances in Neural Information Processing Systems*, Vol. 37, 2024, arXiv:2410.23875.

- [7] L. Luo, Y.-F. Li, G. Haffari, S. Pan, Reasoning on graphs: Faithful and interpretable large language model reasoning, in: International Conference on Learning Representations (ICLR), 2024, arXiv:2310.01061.
- [8] W.-t. Yih, M. Richardson, C. Meek, M.-W. Chang, J. Suh, The value of semantic parse labeling for knowledge base question answering, in: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), 2016, pp. 201–206.
- [9] A. Talmor, J. Berant, The web as a knowledge-base for answering complex questions, in: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2018, pp. 641–651.
- [10] Y. Li, Z. Li, P. Wang, J. Li, X. Sun, H. Cheng, J. X. Yu, A survey of graph meets large language model: Progress and future directions, arXiv preprint arXiv:2311.12399 (2023).
- [11] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, Y. Su, HippoRAG: Neurobiologically inspired long-term memory for large language models, in: Advances in Neural Information Processing Systems, Vol. 37, 2024, pp. 59532–59569.
- [12] D. Li, Y. Niu, Y. Ai, X. Zou, B. Qi, J. Liu, T-GRAG: A dynamic GraphRAG framework for resolving temporal conflicts and redundancy in knowledge retrieval, in: Proceedings of the 33rd ACM International Conference on Multimedia, 2025, pp. 11880–11889.

- [13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.
- [14] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.
- [15] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao, Reflexion: Language agents with verbal reinforcement learning, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.
- [16] J. Jiang, K. Zhou, W. X. Zhao, Y. Song, C. Zhu, H. Zhu, J.-R. Wen, KG-Agent: An efficient autonomous agent framework for complex reasoning over knowledge graph, arXiv preprint arXiv:2402.11163 (2024).
- [17] Y. Xu, S. He, J. Chen, Z. Wang, Y. Song, H. Tong, K. Liu, J. Zhao, Generate-on-graph: Treat LLM as both agent and KG for incomplete knowledge graph question answering, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024, arXiv:2404.14741.
- [18] J. Jiang, K. Zhou, Z. Dong, K. Ye, W. X. Zhao, J.-R. Wen, StructGPT: A general framework for large language models to reason over structured data, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, arXiv:2305.09645.

- [19] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, X. Wu, Unifying large language models and knowledge graphs: A roadmap, *IEEE Transactions on Knowledge and Data Engineering* 36 (7) (2024) 3580–3599.
- [20] C. Ma, Y. Chen, T. Wu, A. Khan, H. Wang, Unifying large language models and knowledge graphs for question answering: Recent advances and opportunities, in: *Proceedings of the 28th International Conference on Extending Database Technology (EDBT)*, 2025, pp. 1174–1177.
- [21] H. Luo, H. E, Y. Guo, Q. Lin, X. Wu, X. Mu, W. Liu, M. Song, Y. Zhu, L. A. Tuan, KBQA-o1: Agentic knowledge base question answering with monte carlo tree search, in: *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, arXiv:2501.18922.
- [22] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, J. Weston, Chain-of-verification reduces hallucination in large language models, in: *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 3563–3578.
- [23] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch, Improving factuality and reasoning in language models through multiagent debate, in: *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [24] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, D. Zhou, Large language models cannot self-correct reasoning yet, in: *International Conference on Learning Representations (ICLR)*, 2024.

- [25] Y. Hu, W. Xuan, Q. Zhou, Z. Li, Y. Li, J. Hu, F. Fang, A self-correcting agentic graph RAG for clinical decision support in hepatology, *Frontiers in Medicine* 12 (2025) 1716327.
- [26] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, H. Hajishirzi, FActScore: Fine-grained atomic evaluation of factual precision in long form text generation, in: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 12076–12100.